3. The *output console*, just below the editor window, is where you can view the transactions and see confirmation of their recording, as well as any errors and debugging details.
4. The *toolchain* on the left panel offers icons representing commands to compile and deploy the compiled code. After clicking the Compile and Deploy icons, click the Deploy button to deploy a smart contract.
5. The *blockchain simulator* offers an execution environment (JavaScript VM) as well as connections to live blockchain networks. The Remix IDE provides a set of account addresses and identities for the test blockchain. The account numbers identify participants. Few (10) accounts are available for testing purposes, but you can create more if you need them.
6. The *user interaction panel*, in the bottom-left corner, lets you interact with a deployed smart contract to run transactions. It exposes all the public functions and data, along with buttons to invoke the functions and text boxes for input parameters. The output of the invocation, if any, is displayed just below the function buttons.
7. All the transactions recorded on the blockchain are provisioned in a .json file for easy review. You can see the Transactions Recorded button in the middle of the panel on the left.

Now you are ready to test the smart contract code that you copied into Remix. Working with the simple Counter.sol contract will enable you to understand the smart contract's